Présentation du TP

Objectifs pédagogiques

À l'issue de ce TP, vous serez capable de :

- écrire des **Dockerfiles** optimisés (cache des couches, build **multi-stage**, HEALTHCHECK, sécurité) ;
- orchestrer une application multi-conteneurs avec **Docker Compose** (réseaux, volumes, variables, dépendances conditionnelles) ;
- construire un **cluster Docker Swarm** de deux nœuds et partager les images entre les nœuds via **Docker Hub** ;
- déployer une **stack** en production : réplicas, contraintes de placement, **secrets**, **configs**, réseaux **overlay**, routing mesh ;
- exploiter le cluster : mise à l'échelle, **rolling update**, **rollback** automatique, maintenance d'un nœud, tolérance aux pannes ;
- analyser les limites d'un déploiement d'éléments **stateful** (base de données) dans un orchestrateur.

L'application TaskBoard

TaskBoard est un gestionnaire de tâches composé de **4 services** :

Service	Technologie	Rôle
frontend	React 19 (Vite) servi par nginx	Interface web + reverse proxy `/api/*` → `backend:5000`
backend	Python 3.13, Flask, gunicorn	API REST, port 5000
db	MySQL 8.4	Stockage persistant des tâches
redis	Redis 7.4	Cache de la liste des tâches (30 s) + compteur de visites

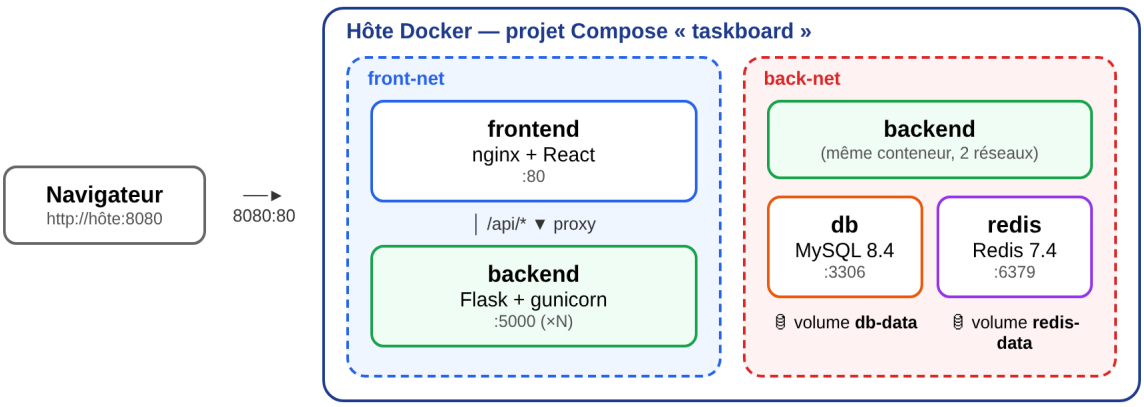


Figure 1 — Architecture cible avec Docker Compose (un seul hôte)

Le pied de page de l'interface affiche le **hostname du conteneur backend** qui a répondu, la **version** de l'API, le **compteur de visites** (stocké dans Redis) et la **source** de la liste (`db` ou `cache`). Ces informations vous serviront à observer la répartition de charge et les mises à jour.

Kit fourni: https://github.com/jadliaissam-edu/taskboard-2.git

```
taskboard/
├── README.md            # description de l'app, variables d'environnement, API
├── backend/
│   ├── app.py           # API Flask
│   └── requirements.txt
├── frontend/
│   ├── package.json, package-lock.json, vite.config.js, index.html
│   ├── src/             # code React
│   └── nginx.conf       # sert dist/ + proxy /api → backend:5000
└── db/
    └── init.sql         # schéma + 3 tâches initiales
```

Lisez attentivement le README.md : il décrit toutes les variables d'environnement attendues par le backend, notamment le mécanisme `VAR_FILE` (ex. `DB_PASSWORD_FILE`) qui permet de lire une valeur depuis un fichier — indispensable pour les secrets Swarm.

Règle — Vous ne devez **pas modifier** le code applicatif (`app.py`, `src/`, `nginx.conf`, `init.sql`). Tout le travail porte sur les fichiers Docker que vous allez créer.

Partie 1 — Conteneurisation

Sécurité — Vos fichiers doivent respecter les **bonnes pratiques de sécurité** Docker : à vous de les identifier et de les appliquer.

1.1 Image du backend

Créez `backend/Dockerfile` respectant **toutes** les contraintes suivantes :

- image de base `python:3.13-slim` ;
- variables `PYTHONUNBUFFERED=1` (logs non bufferisés) et `PIP_NO_CACHE_DIR=1` ;
- installation des dépendances **avant** la copie du code source, afin de profiter du cache de build ;
- un argument de build `APP_VERSION` (valeur par défaut `1.0`) exposé au runtime via la variable d'environnement `APP_VERSION` ;
- port documenté : `5000` ;
- un `HEALTHCHECK` interrogeant `http://localhost:5000/api/health` — attention, `curl` **n'est pas présent** dans l'image slim : utilisez Python ;
- commande de démarrage : `gunicorn` sur `0.0.0.0:5000` avec 2 workers et les logs d'accès sur la sortie standard.

Ajoutez un fichier `backend/.dockerignore` adapté.

- Construisez l'image `taskboard-backend:1.0` et vérifiez sa taille.

1.2 Image du frontend (multi-stage)

Créez `frontend/Dockerfile` en **deux étapes** :

1. **Étape « build »** basée sur `node:22-alpine` : copie de `package.json` et `package-lock.json`, `npm ci`, copie du reste des sources, `npm run build` (produit le dossier `dist/`).
2. **Étape finale** basée sur `nginx:1.29-alpine` : copie de `nginx.conf` vers `/etc/nginx/conf.d/default.conf` et du dossier `dist/` de l'étape précédente vers `/usr/share/nginx/html`. Ajoutez un `HEALTHCHECK` (indice : `wget` est disponible dans l'image alpine).

Ajoutez `frontend/.dockerignore` (au minimum `node_modules` et `dist`).

- Construisez l'image `taskboard-frontend:1.0`.
- Construisez également l'étape intermédiaire seule (option `--target`) sous le tag `taskboard-frontend:build` et comparez la taille des deux images.

Partie 2 — Docker Compose

> **Sécurité —** Vos fichiers doivent respecter les **bonnes pratiques de sécurité** Docker : à vous de les identifier et de les appliquer.

2.1 Fichier .env

Créez un fichier `.env` à la racine du projet :

```
MYSQL_ROOT_PASSWORD=rootpass
DB_NAME=taskboard
DB_USER=taskboard
DB_PASSWORD=taskpass
FRONT_PORT=8080
```

2.2 Fichier compose.yaml

Écrivez `compose.yaml` (nom de projet : `taskboard`) décrivant les 4 services avec les contraintes suivantes :

Réseaux et exposition

- deux réseaux : `front-net` (frontend, backend) et `back-net` (backend, db, redis) ;
- seul le **frontend** publie un port sur l'hôte : `${FRONT_PORT}` → 80.

Services

- **frontend** et **backend** : construits depuis leurs dossiers respectifs (`build`) et tagués `taskboard-frontend:1.0` / `taskboard-backend:1.0` (`image`) ;
- **backend** : variables `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `REDIS_HOST` alimentées à partir du `.env` (interpolation `${…}`) ;
- **db** : image `mysql:8.4`, variables `MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE`, `MYSQL_USER`, `MYSQL_PASSWORD` ; volume nommé `db-data` sur `/var/lib/mysql` ; `db/init.sql` monté dans `/docker-entrypoint-initdb.d/` ;
- **redis** : image `redis:7.4-alpine`, persistance AOF (`redis-server --appendonly yes`), volume nommé `redis-data` sur `/data` ;
- politique de redémarrage `unless-stopped` pour tous les services.

Santé et ordre de démarrage

- **healthcheck** pour `db` (`mysqladmin ping -h 127.0.0.1 …`) et `redis` (`redis-cli ping`) ; prévoir un `start_period` suffisant pour MySQL ;
- `backend` ne démarre que lorsque `db` et `redis` sont **healthy** ; `frontend` ne démarre que lorsque `backend` est **healthy** (`depends_on` + `condition`).

2.3 Lancement et tests

- Validez le fichier et affichez sa version interpolée.
- Construisez et démarrez le projet en arrière-plan.
- Vérifiez que les 4 services sont **healthy** et consultez les logs du backend.
- Interrogez `/api/health` et `/api/tasks` sur le port 8080.

- Ouvrez `http://IP_VM1:8080` dans un navigateur : ajoutez, cochez et supprimez des tâches.

2.4 Expérimentations

a) Cache Redis

- Appelez `/api/tasks` deux fois de suite et observez le champ `source`.
- Depuis le conteneur `redis`, listez les clés présentes et affichez le TTL de la clé `tasks:all`.
- Ajoutez une tâche puis rappelez `/api/tasks`.

b) Isolation réseau et DNS

- Depuis le conteneur `frontend`, appelez `http://backend:5000/api/health`, puis tentez de résoudre le nom `db`.
- Depuis le conteneur `backend`, résolvez le nom `db`.
- Listez les réseaux créés par le projet.

c) Persistance

- Ajoutez une tâche « persistance » via l'interface.
- Arrêtez et supprimez le projet **sans** les volumes, puis relancez-le : vérifiez la présence de la tâche.
- Recommencez en supprimant **aussi** les volumes.

d) Mise à l'échelle

- Passez le service `backend` à **3** instances.
- Appelez `/api/info` plusieurs fois et observez le champ `hostname`.
- Tentez de passer le `frontend` à 2 instances.

Arrêtez le projet Compose avant la partie 3 (conservez les images).

Partie 3 — Construction du cluster Swarm

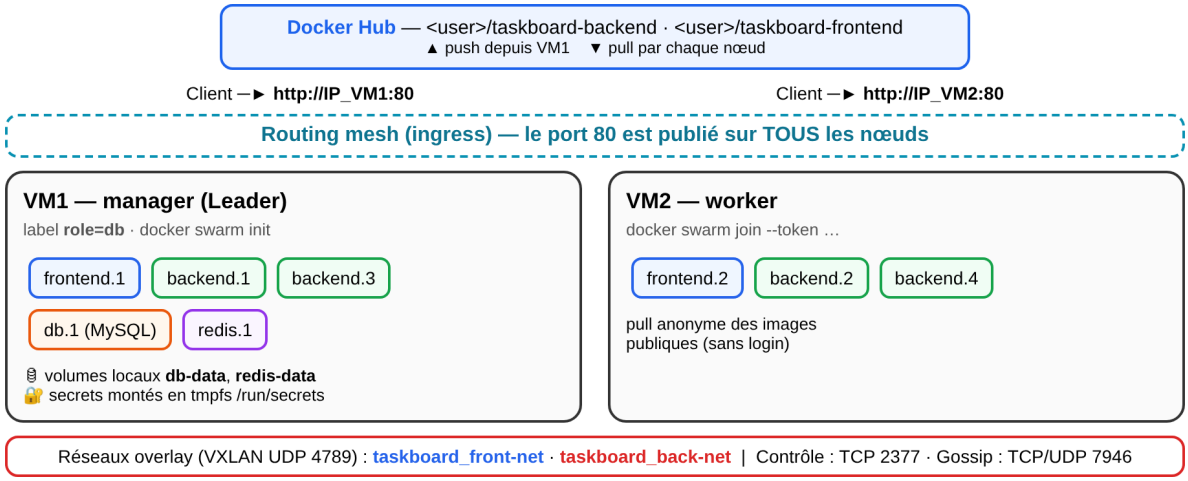


Figure 2 — Déploiement cible sur le cluster Swarm à deux nœuds

3.1 Ports utilisés par Swarm

Les deux machines cloud ont tous leurs ports ouverts. Pour information, Swarm utilise :

Port	Protocole	Usage
2377	TCP	Gestion du cluster (communication avec les managers)

Port	Protocole	Usage
7946	TCP + UDP	Découverte des nœuds (gossip)
4789	UDP	Trafic des réseaux overlay (VXLAN)

> **À savoir —** Si vos VM disposent d'une IP privée et d'une IP publique, utilisez l'**IP privée** pour initialiser et rejoindre le cluster, et l'**IP publique** pour accéder à l'application depuis votre navigateur.

3.2 Initialisation

- Initialisez le cluster sur **VM1** (manager) en précisant l'adresse annoncée.
- Faites rejoindre **VM2** au cluster en tant que **worker**.
- Depuis VM1, listez les nœuds et inspectez le nœud courant.

3.3 Labels de nœuds

La base de données et Redis utilisent des **volumes locaux** : ils doivent toujours être planifiés sur le même nœud. Posez le label `role=db` sur **VM1** et vérifiez-le.

3.4 Partage des images via Docker Hub

VM2 ne possède pas les images construites sur VM1 : chaque nœud doit pouvoir les **télécharger depuis un registre**. Vous utiliserez **Docker Hub**.

1. Créez un compte gratuit sur **hub.docker.com** (si vous n'en avez pas).
2. Connectez-vous à Docker Hub **sur VM1 uniquement** (nécessaire pour pousser) avec votre compte.
3. Depuis VM1, taguez les deux images sous la forme `<identifiant>/taskboard-backend:1.0` et `<identifiant>/taskboard-frontend:1.0`, puis poussez-les.
4. Sur hub.docker.com, vérifiez que les deux dépôts sont créés et que leur visibilité est **Public** (*Settings → Visibility*).
5. Sur **VM2**, sans aucune authentification, téléchargez l'image backend.

> **À savoir —** Les images étant **publiques**, les nœuds du cluster les téléchargent sans authentification : aucun `docker login` n'est nécessaire sur VM2.

Partie 4 — Déploiement de la stack

4.1 Secrets

Créez deux **secrets** Swarm, `db_root_password` et `db_password`, à partir de l'entrée standard, puis listez-les et inspectez-en un.

4.2 Fichier stack.yaml

> **Sécurité —** Vos fichiers doivent respecter les **bonnes pratiques de sécurité** Docker : à vous de les identifier et de les appliquer.

Écrivez `stack.yaml` en partant de votre `compose.yaml`. Contraintes :

Généralités

- les directives `build`, `depends_on` (conditions) et `restart` ne sont **pas prises en compte** par `docker stack deploy` : retirez-les ; les images proviennent de Docker Hub : `${DOCKERHUB_USER}/taskboard-…:1.0` (la variable est interpolée depuis le shell au moment du déploiement) ;
- réseaux `front-net` et `back-net` en driver **overlay** ;
- le secret et la config sont déclarés au niveau racine : secrets `external: true`, config `db_init` créée depuis `./db/init.sql`.

frontend

- **2 réplicas**, port **80** publié ;
- mise à jour progressive : 1 tâche à la fois, ordre **start-first**, délai 5 s.

backend

- **4 réplicas** ;
- mot de passe lu depuis le secret : `DB_PASSWORD_FILE=/run/secrets/db_password` ;
- mise à jour : `parallelism: 1`, `delay: 10s`, `order: start-first`, `monitor: 30s`, `failure_action: rollback` ;
- redémarrage en cas d'échec (`restart_policy`).

db

- **1 réplica** placé sur le nœud `node.labels.role == db` ;
- mots de passe via les secrets et les variables `MYSQL_ROOT_PASSWORD_FILE` / `MYSQL_PASSWORD_FILE` ;
- `init.sql` fourni via la **config** `db_init` (cible `/docker-entrypoint-initdb.d/init.sql`) ;
- healthcheck adapté : le mot de passe root doit être lu depuis `/run/secrets/db_root_password` (pensez à l'échappement `$$` dans le YAML).

redis

- **1 réplica** placé sur le même nœud que `db` ;
- **healthcheck** (`redis-cli ping`).

4.3 Déploiement

- Définissez la variable `DOCKERHUB_USER` dans votre shell.
- Déployez la stack sous le nom `taskboard`.
- Vérifiez que les dépendances sont respectées : `db` et `redis` doivent être **healthy** avant que les tâches `backend` ne se stabilisent, et le `backend` avant que le `frontend` ne serve l'API. En cas de problème, déployez en deux temps (d'abord `db` et `redis`, puis le reste).
- Listez ses services et ses tâches (nœud, état, erreur éventuelle) ; suivez les logs du backend.

> **Normal —** Au premier déploiement, MySQL met 20 à 40 s à s'initialiser. Il est **normal** que quelques tâches backend redémarrent avant de devenir stables.

4.4 Vérifications

- Appelez `/api/info` plusieurs fois via `IP_VM1` **puis** via `IP_VM2` et observez le champ `hostname`.
- Ouvrez l'interface via `http://IP_VM1` **et** via `http://IP_VM2`.
- Dans un conteneur backend, affichez le contenu de `/run/secrets` et les variables `DB_*`.

Partie 5 — Exploitation du cluster

5.1 Mise à l'échelle

Passez le backend à **6** réplicas, observez leur répartition sur les nœuds, puis revenez à **4**.

5.2 Rolling update

- Construisez une version **1.1** du backend **sans modifier le code** (argument de build `APP_VERSION`) et poussez-la sur Docker Hub.
- Mettez à jour l'image du service backend avec cette version.
- Pendant la mise à jour, suivez en continu l'état des tâches dans un terminal et appelez `/api/info` en boucle dans un second terminal.
- Testez ensuite le **retour arrière manuel**, vérifiez la version affichée, puis réappliquez la version 1.1.

5.3 Rollback automatique

- Simulez un déploiement défectueux en ajoutant la variable `FAIL_HEALTH=1` au service backend (elle fait échouer `/api/health`).
- Suivez l'état des tâches puis consultez le statut de mise à jour (`UpdateStatus`) du service.

5.4 Maintenance d'un nœud

- Passez **VM2** en mode **drain** et observez où tournent les tâches.
- Repassez VM2 en mode **active** et observez à nouveau.
- Forcez la redistribution des tâches du backend.

5.5 Panne d'un nœud

- Sur **VM2**, arrêtez le service Docker pour simuler une panne.
- Sur **VM1**, observez pendant environ une minute l'état des nœuds et des tâches.
- Redémarrez Docker sur VM2.

5.6 Nettoyage

Supprimez la stack et les secrets, faites quitter le cluster aux deux nœuds, puis déconnectez-vous de Docker Hub sur VM1.